

AI Smart Campus System

Backend Project Documentation

Course Name	Mobile Computing Lab
Course Code	CSE4204
Submitted To	Md. Riaz Mahmud
Team Number	T07
Department	Computer Science & Engineering
University	Northern University of Business and Technology, Khulna
Submission Date	June 29, 2026

2. Team Information

Team Number: T07

Project Name: AI Smart Campus System

Section: 8A

Course: CSE4204 — Mobile Computing Lab

Instructor: Md. Riaz Mahmud

#	Member Name	Student ID	Role
1	Md. Nazmus Shakib	11220320852	Database (Team Leader)
2	Samira Akter Mitu	11220320858	AI Integration Developer
3	Md. Tanvin Sadik Dhrubo	11220320860	Frontend Developer
4	Khan Waziur Rahman	11220320861	Backend Developer

3. Project Overview

3.1 Objective

The primary objective of this project is to build a centralized, intelligent campus management system that streamlines administrative tasks, student records, and resource management. The system is designed to minimize manual intervention through smart automation and AI-powered recommendations, creating a modern digital campus environment.

3.2 Description

The AI Smart Campus System is a robust backend solution developed using Laravel 12. It provides secure, well-structured RESTful APIs for managing campus operations, including task scheduling, AI-driven academic recommendations, and learning resource distribution. The backend is built following industry-standard practices with token-based authentication, request validation, and modular service architecture.

3.3 User Roles

- Student — Access learning resources, view AI-generated recommendations, and manage personal tasks.
- Faculty — Upload and manage learning materials, assign tasks, and monitor student progress.
- Admin — Full system access including user management, system configuration, and analytics.

4. Backend Technology Stack

The following technologies have been selected to ensure a scalable, maintainable, and secure backend infrastructure for the AI Smart Campus System.

Component	Technology
Backend Framework	Laravel 12
Programming Language	PHP 8.2
Database	MySQL
Authentication	Laravel Sanctum (Token-Based)
API Testing Tool	Postman
Version Control	Git / GitHub

The GitHub repository serves as the central version control hub for all backend source code, enabling collaborative development and continuous tracking of changes across the development lifecycle.

5. Backend Project Setup

The following steps were carried out to initialize and configure the backend development environment for the AI Smart Campus System.

- Laravel 12 project initialized using Composer CLI.
- All required Composer dependencies installed and composer.lock committed to the repository.
- Environment configuration file (.env) set up with MySQL database credentials, app key, and debug settings.
- Laravel Sanctum installed and configured for token-based API authentication.
- API routes defined and organized within routes/api.php following RESTful conventions.
- Database migrations created and executed to generate all required tables.
- Git repository initialized and synchronized with GitHub remote.

6. Database Design Summary

Database Name: campus_management

6.1 Core Tables

- users — Stores all registered users with role designation (student, faculty, admin), hashed passwords, and profile data.
- tasks — Manages tasks assigned to or created by users, with status tracking and foreign key linkage to users.
- recommendations — Stores AI-generated and manually added academic recommendations linked to individual users.
- learning_resources — Contains learning materials uploaded by faculty, categorized and accessible to students.

6.2 Relationships

The database schema follows a normalized design with One-to-Many relationships between the Users table and both the Tasks and Learning Resources tables. Each user can have multiple tasks and access multiple resources, while each task and resource belongs to exactly one user.

6.3 Validation

Strict validation rules have been implemented using Laravel's Form Request classes. All incoming data is validated before processing, ensuring data integrity, preventing injection attacks, and returning structured error responses for invalid inputs.

6.4 Databases Table

The screenshot shows the phpMyAdmin interface for a database named 'laravel'. The left sidebar lists various tables including 'academic_records', 'admins', 'assessment_records', 'attendance_records', 'cache', 'cache_locks', 'courses', 'course_recommendations', 'faculty', 'failed_jobs', 'jobs', 'job_batches', 'migrations', 'notifications', 'password_reset_tokens', 'performance_metrics', 'risk_alerts', 'sessions', 'students', 'users', 'library_management', 'mysq', 'performance_schema', 'phpmyadmin', 'startup', and 'test'. The main panel displays a table structure for 'laravel' with columns: Table, Action, Rows, Type, Collation, Size, and Overhead. The table 'laravel' has 20 tables and a sum of 14 InnoDB utf8mb4_general_ci 512.0 K18.

Table	Action	Rows	Type	Collation	Size	Overhead
academic_records	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 K18	-
admins	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 K18	-
assessment_records	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	48.0 K18	-
attendance_records	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 K18	-
cache	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 K18	-
cache_locks	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 K18	-
courses	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 K18	-
course_recommendations	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 K18	-
faculty	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 K18	-
failed_jobs	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 K18	-
jobs	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 K18	-
job_batches	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 K18	-
migrations	Browse Structure Search Insert Empty Drop	14	InnoDB	utf8mb4_unicode_ci	16.0 K18	-
notifications	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 K18	-
password_reset_tokens	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 K18	-
performance_metrics	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 K18	-
risk_alerts	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 K18	-
sessions	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	48.0 K18	-
students	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 K18	-
users	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 K18	-
Sum		14	InnoDB	utf8mb4_general_ci	512.0 K18	0.0

Figure 6.1: Database Tables of the AI Smart Campus System

6.5 User Table

The screenshot shows the phpMyAdmin interface for the 'users' table in the 'ai_smart_campus' database. The table structure is displayed with columns: #, Name, Type, Collation, Attributes, Null, Default, Comments, Extra, and Action. The table has 10 columns: id (bigint(20), UNSIGNED, NULL, None, AUTO_INCREMENT), name (varchar(255), utf8mb4_unicode_ci, NULL, None), email (varchar(255), utf8mb4_unicode_ci, NULL, None), email_verified_at (timestamp, utf8mb4_unicode_ci, Yes, NULL), password (varchar(255), utf8mb4_unicode_ci, NULL, None), role (enum('student', 'faculty', 'admin'), utf8mb4_unicode_ci, No, student), approval_status (enum('pending', 'approved', 'rejected'), utf8mb4_unicode_ci, No, pending), remember_token (varchar(100), utf8mb4_unicode_ci, Yes, NULL), created_at (timestamp, Yes, NULL), and updated_at (timestamp, Yes, NULL). The table also has two indexes: a PRIMARY index on 'id' and a BTREE index on 'email'.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	id	bigint(20)		UNSIGNED	No	None		AUTO_INCREMENT	Change Drop More
2	name	varchar(255)	utf8mb4_unicode_ci		No	None			Change Drop More
3	email	varchar(255)	utf8mb4_unicode_ci		No	None			Change Drop More
4	email_verified_at	timestamp			Yes	NULL			Change Drop More
5	password	varchar(255)	utf8mb4_unicode_ci		No	None			Change Drop More
6	role	enum('student', 'faculty', 'admin')	utf8mb4_unicode_ci		No	student			Change Drop More
7	approval_status	enum('pending', 'approved', 'rejected')	utf8mb4_unicode_ci		No	pending			Change Drop More
8	remember_token	varchar(100)	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
9	created_at	timestamp			Yes	NULL			Change Drop More
10	updated_at	timestamp			Yes	NULL			Change Drop More

Figure 6.2: Users Table

6.6 Personal Access Tokens Table

The screenshot shows the phpMyAdmin interface for the 'personal_access_tokens' table. The table has the following structure:

id	tokenable_type	tokenable_id	name	token	abilities	last_used_at	expires_at	created_at	updated_at
1	App\Models\User	1	auth_token	bb33e945f15bec0676095f947d868f29d9552a9e6c046471...	["*"]	NULL	NULL	2026-06-29 04:33:34	2026-06-29 04:33:34
2	App\Models\User	1	auth_token	4c3017a821fd20f85e2e3a29aa0c918f10155ef3438d2137...	["*"]	NULL	NULL	2026-06-29 04:41:39	2026-06-29 04:41:39

Figure 6.3: Personal Access Tokens Table

6.7 Tasks Table

The screenshot shows the phpMyAdmin interface for the 'tasks' table. The table has the following structure:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	id	bigint(20)		UNSIGNED	No	None		AUTO_INCREMENT	Change Drop More
2	title	varchar(255)	utf8mb4_unicode_ci		No	None			Change Drop More
3	description	text	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
4	assigned_to	varchar(255)	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
5	due_date	date			Yes	NULL			Change Drop More
6	status	enum('pending', 'in_progress', 'completed')	utf8mb4_unicode_ci		No	pending			Change Drop More
7	priority	enum('low', 'medium', 'high')	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
8	created_at	timestamp			Yes	NULL			Change Drop More
9	updated_at	timestamp			Yes	NULL			Change Drop More

Figure 6.4: Tasks Table

6.8 Recommendations Table

localhost/phpmyadmin/index.php?route=/table/structure&db=laravel&table=recommendations

Table structure

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	id	bigint(20)		UNSIGNED	No	None		AUTO_INCREMENT	Change Drop More
2	title	varchar(255)	utf8mb4_unicode_ci		No	None			Change Drop More
3	description	text	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
4	recommendation_type	varchar(255)	utf8mb4_unicode_ci		No	None			Change Drop More
5	target_user	varchar(255)	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
6	score	double			Yes	NULL			Change Drop More
7	created_at	timestamp			Yes	NULL			Change Drop More
8	updated_at	timestamp			Yes	NULL			Change Drop More

Indexes

Action	Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
Edit Rename Drop	PRIMARY	BTREE	Yes	No	id	0	A	No	

Create an index on 1 columns Go

Partitions

No partitioning defined!

Activate Windows
Go to Settings to activate Windows.

Figure 6.5: Recommendations Table

6.9 Learning Resources Table

localhost/phpmyadmin/index.php?route=/table/structure&db=laravel&table=learning_resources

Table structure

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	id	bigint(20)		UNSIGNED	No	None		AUTO_INCREMENT	Change Drop More
2	title	varchar(255)	utf8mb4_unicode_ci		No	None			Change Drop More
3	description	text	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
4	category	varchar(255)	utf8mb4_unicode_ci		No	None			Change Drop More
5	resource_type	varchar(255)	utf8mb4_unicode_ci		No	None			Change Drop More
6	resource_url	varchar(255)	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
7	uploaded_by	varchar(255)	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
8	created_at	timestamp			Yes	NULL			Change Drop More
9	updated_at	timestamp			Yes	NULL			Change Drop More

Indexes

Action	Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
Edit Rename Drop	PRIMARY	BTREE	Yes	No	id	0	A	No	

Create an index on 1 columns Go

Partitions

No partitioning defined!

Activate Windows
Go to Settings to activate Windows.

Figure 6.4: Learning Resources Database Table

7. Authentication System

The authentication system has been implemented using Laravel Sanctum, providing a lightweight and secure token-based mechanism suitable for Single Page Applications (SPAs) and mobile clients.

- User Registration — New users can register with validated credentials. Passwords are securely hashed using bcrypt before storage.
- User Login — Authenticated users receive a personal access token upon successful login, which must be included in all subsequent API requests.
- User Logout — Tokens are revoked server-side upon logout, ensuring stateless and secure session termination.
- Token-Based Session Management — All protected routes require a valid Bearer token in the Authorization header, enabling stateless API interactions.
- Role-Based Access Control — Middleware has been implemented to enforce role-specific access, ensuring that students, faculty, and admins can only access their permitted resources.

7.1 Registration API Test

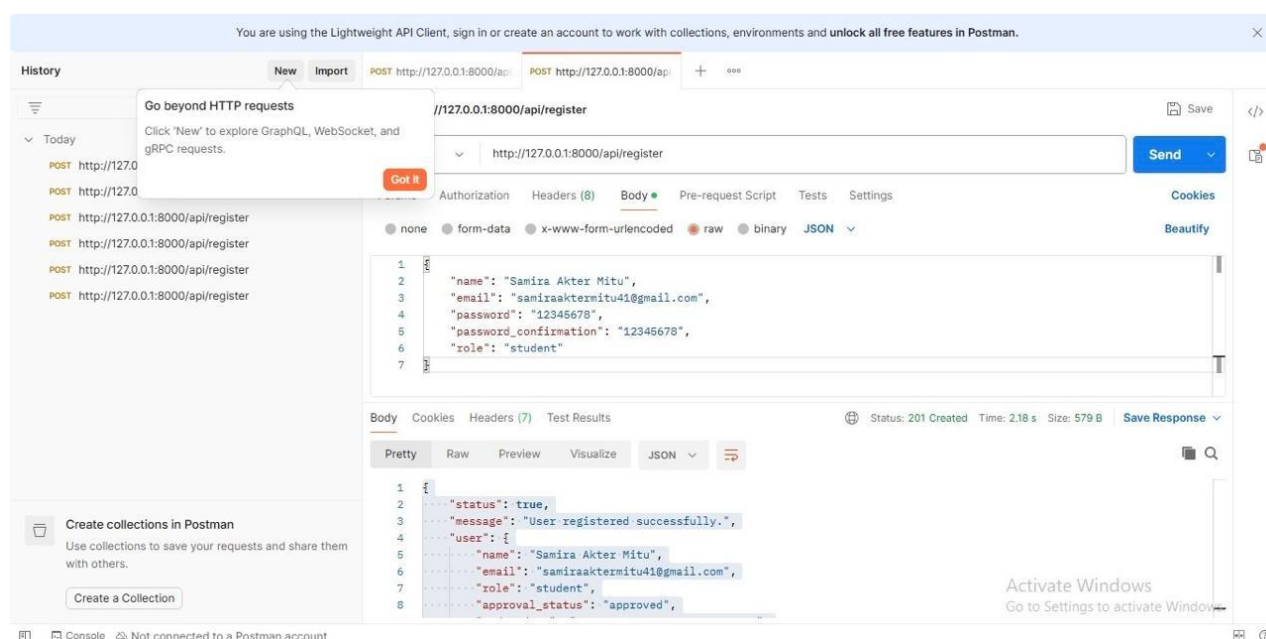


Figure 7.1: Registration API tested successfully using Postman.

7.2 Login API Test

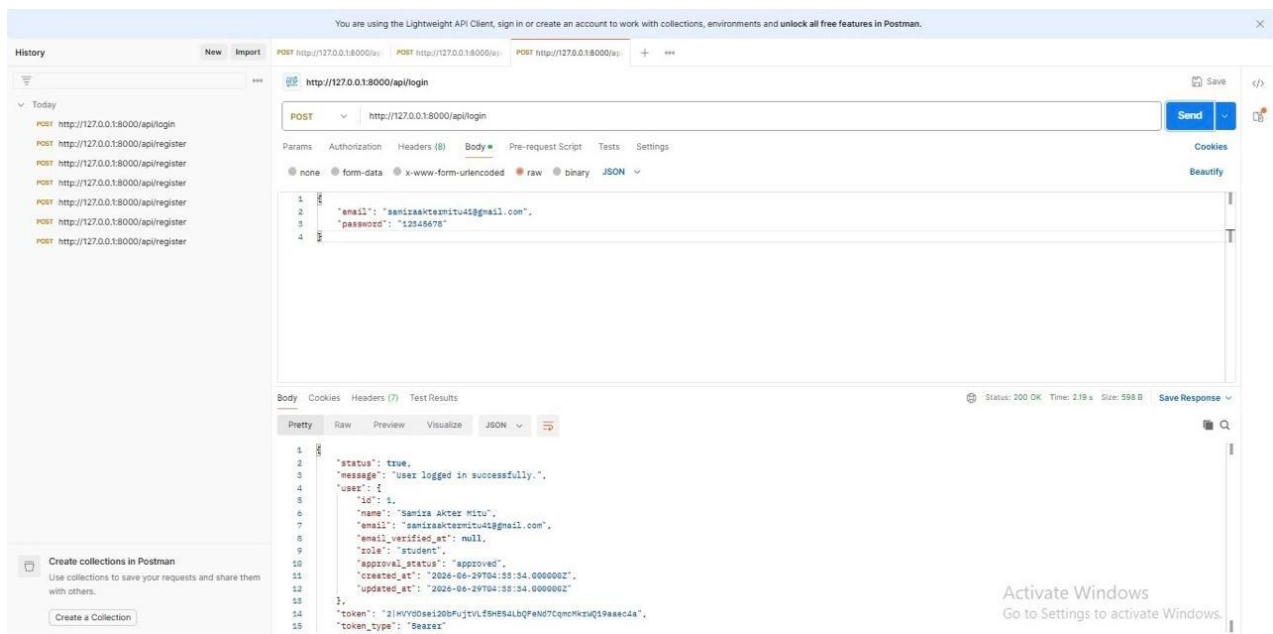


Figure 7.2: Login API tested successfully.

7.3 Profile API Test

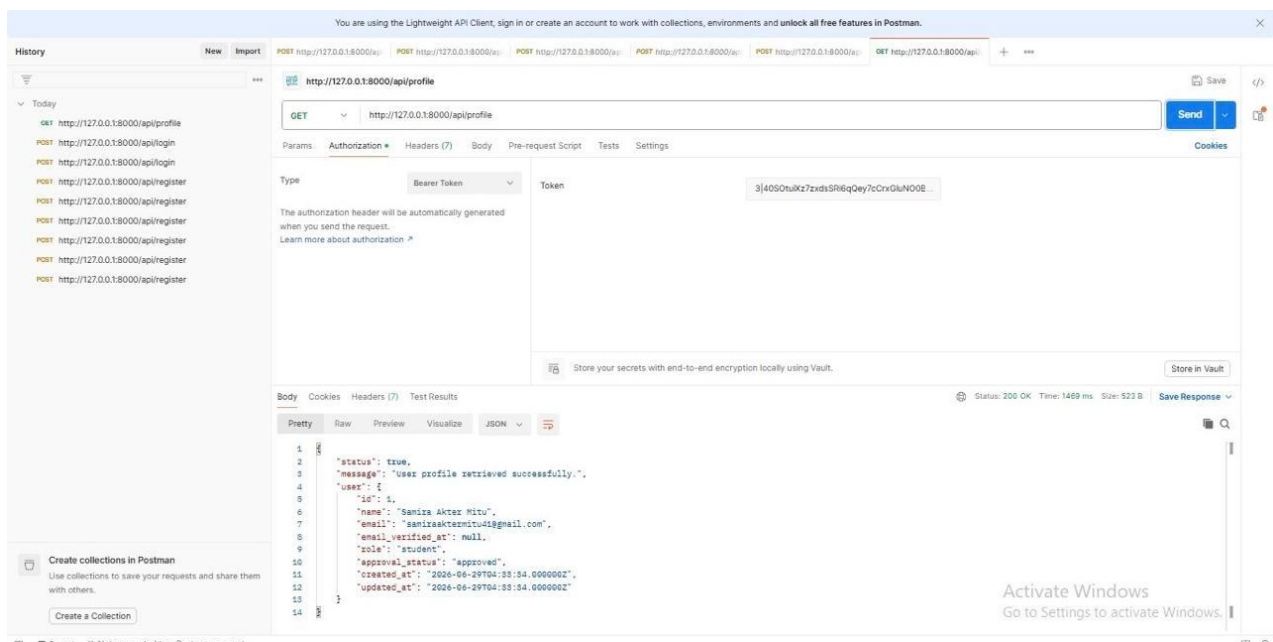


Figure 7.3: Authenticated user profile retrieved successfully.

7.4 Logout API Test

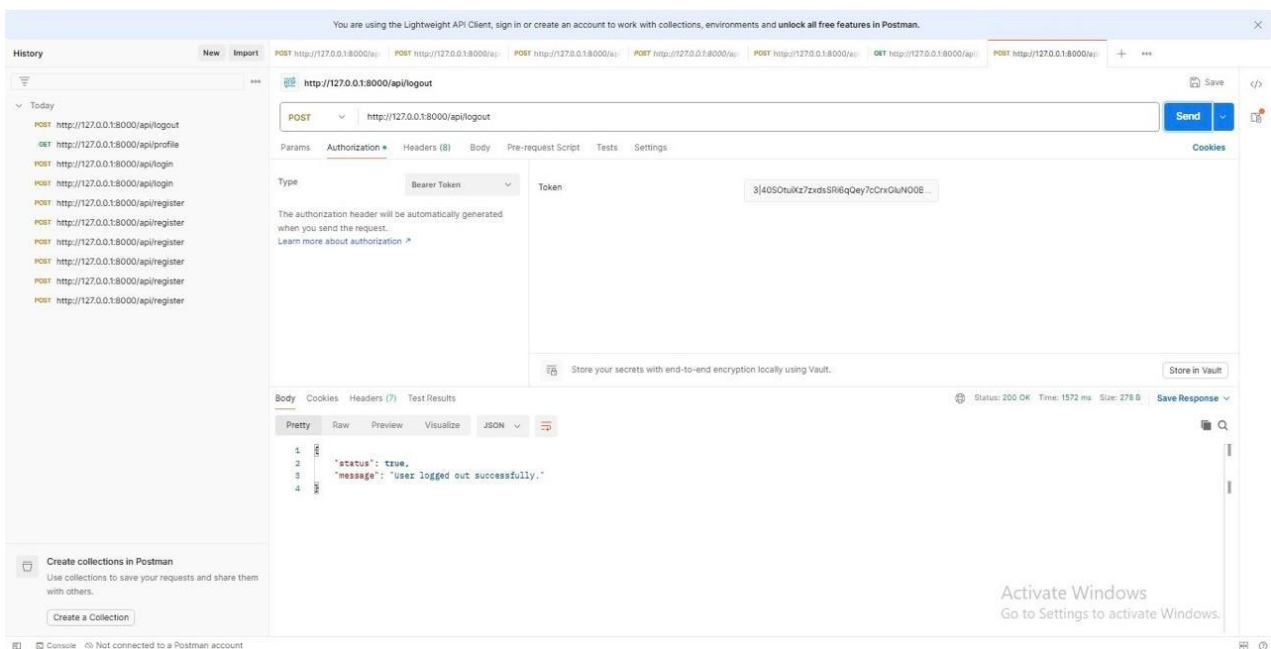


Figure 7.4: User logged out successfully.

7.5 Validation/ Error Handling

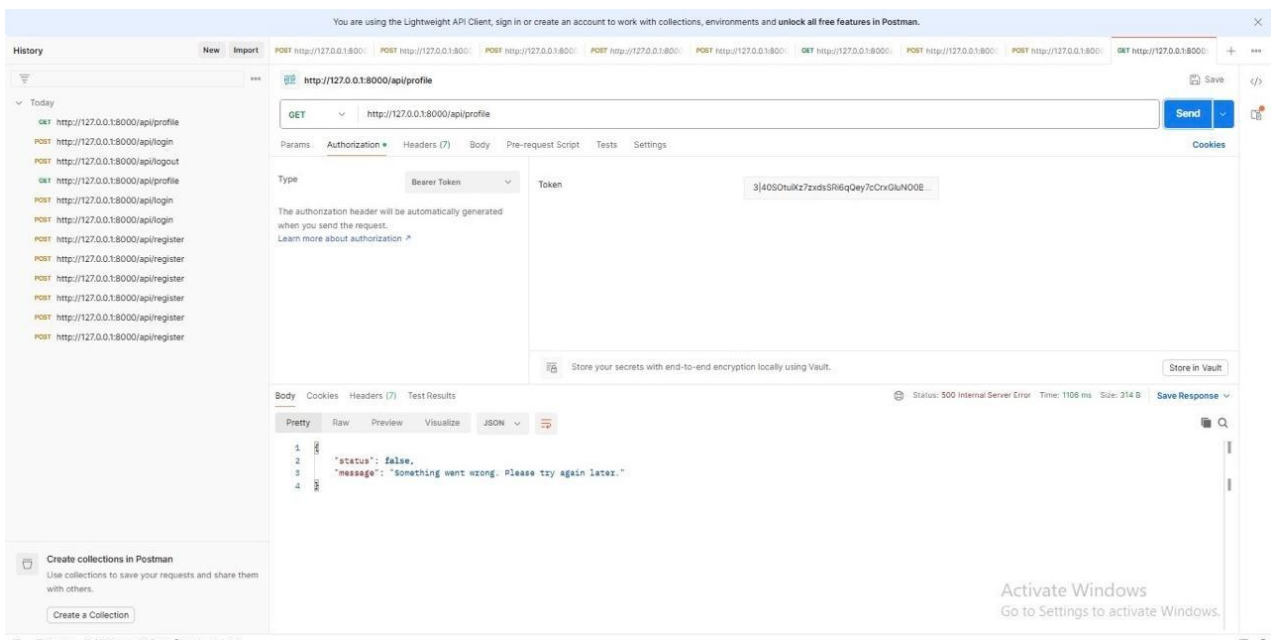


Figure 7.5: Validation error response for invalid input.

8. Implemented APIs

The following RESTful API endpoints have been developed and tested. All routes are prefixed with /api and require appropriate authentication tokens unless specified otherwise.

Method	Endpoint	Description
POST	/api/register	User Registration
POST	/api/login	User Login
POST	/api/logout	User Logout
GET	/api/profile	Fetch User Profile
GET	/api/tasks	Retrieve All Tasks
POST	/api/tasks	Create New Task
GET	/api/recommendations	Fetch AI Recommendations
POST	/api/recommendations	Add New Recommendation
GET	/api/learning-resources	View Learning Materials
POST	/api/learning-resources	Add New Learning Material

All endpoints return JSON responses following a consistent structure with status codes, message fields, and data payloads. Error responses include descriptive messages and validation details where applicable.

9. Current Development Progress

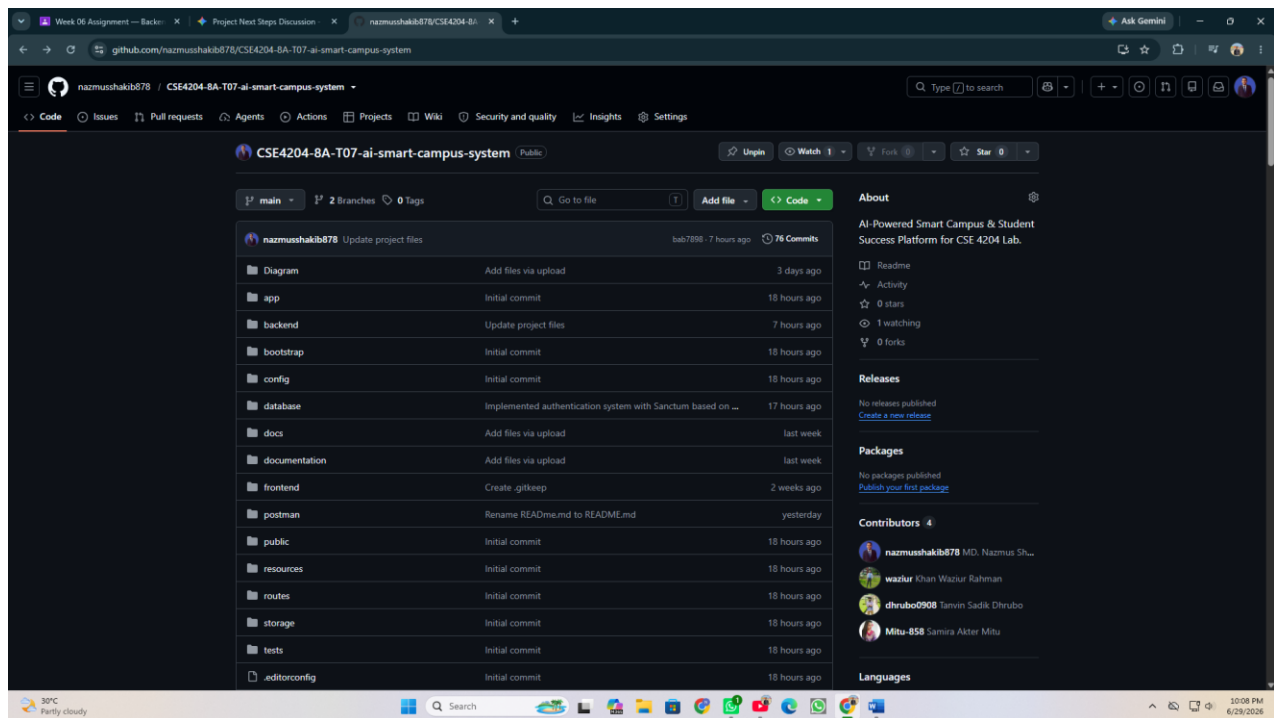
As of the current submission date, the following development milestones have been successfully completed for the AI Smart Campus System backend.

- Backend architecture setup completed — Laravel 12 project structure organized following MVC pattern with service and repository layers.
- Database schema and migrations finalized — All core tables created with proper relationships, indexes, and constraints.
- Authentication system fully functional — Registration, login, logout, and token management tested via Postman.
- Core business logic APIs developed — Task management, recommendations, and learning resource endpoints implemented and operational.
- Error handling and validation implemented — Global exception handler configured; all requests validated through Form Request classes.
- GitHub repository synchronized — All commits pushed with meaningful commit messages; repository is up to date.

10. GitHub Repository

The complete source code and version control history of this backend project are maintained in our GitHub repository for collaboration and project management.

Project Repository (GitHub): [CSE4204-8A-T07-AI-Smart-Campus-System Repository](https://github.com/nazmusshakib878/CSE4204-8A-T07-ai-smart-campus-system)



11. Conclusion

The backend infrastructure for the AI Smart Campus System has been successfully designed and implemented using Laravel 12. The system provides a secure, scalable, and efficient foundation for managing campus-related data and operations through well-defined RESTful APIs.

All core authentication flows, database relationships, and API endpoints are fully functional and have been verified through Postman testing. The codebase follows Laravel best practices with strict validation, role-based access control, and modular architecture, ensuring ease of maintenance and future extensibility.

With this backend foundation in place, the project is well-positioned for the subsequent phases of development, including AI integration via the Google Gemini API, advanced analytics, and the deployment of the frontend React.js interface.

— Team T07, CSE4204-8A

Northern University of Business and Technology, Khulna
Submitted: June 29, 2026